

Snowflake Cloud Data Warehouse – Cheat Sheet

Course: HEIG-VD IST 2025/26 | Authors: Markaj Agron, Jorand Yuuta, Stampfli Nathan, Liao Pei-Wen | 12.06.2026

Why Snowflake?

Scenario: SwissBike SA

SwissBike SA is a Swiss SME (50 employees, 1,500 orders/month) selling bicycles online. Data is scattered across e-commerce, ERP, CRM and marketing tools. Analytics rely on manual Excel exports; query performance degrades as data grows; no dedicated data team exists.

SME Pain Point	Snowflake Solution
More analytics users	Add Virtual Warehouses without touching storage
Query speed degrades as data grows	Micro-partitions + pruning skip irrelevant data
Disconnected systems, no single source of truth	Snowpipe centralizes all sources
Manual exports, stale reports	Automated ingestion, always up to date
Budget sensitive	Pay only when warehouse is running

What Snowflake Does

Snowflake is a cloud-native data warehouse using a **hybrid model** (Shared-disk + Shared-nothing + MPP) with three independent layers:

- **Cloud Services** : always-on metadata, query optimization, auth. Free under 10% of daily compute usage.
- **Compute (Virtual Warehouses)** : independent clusters; 24h query result cache -> free repeated queries; auto-scale and pause when idle. Cold start: 2–3s.
- **Storage** : columnar, auto-compressed (3–5×), micro-partitioned (50–500 MB chunks). Pruning skips irrelevant partitions at query time. Decoupled from compute -> pay separately.

Data protection: Time Travel (restore up to 90 days) + Fail-safe (7-day recovery after Time Travel expires).

Inputs: e-commerce, ERP, CRM, CSV/API imports, Snowpipe streaming, external stages (JSON, Parquet, Avro, ORC, XML).

Outputs: SQL analytics, dashboards (Power BI, Tableau, Looker), reports, ML datasets.

Benefits and Limitations

Benefits and Trade-offs by Layer

Layer	Advantage	Trade-off
Architecture	Decoupled compute & storage; MPP -> concurrent users, no interference	Cold start 2–3s; not for OLTP (INSERT / UPDATE slower than PostgreSQL)
Compute	Pay per second, zero cost when suspended; 24h result cache -> free repeated queries	No hard spend cap by default
Storage	3–5× columnar compression; pruning keeps query speed constant as data grows; Time Travel + Fail-safe (90d + 7d)	Most expensive EU storage (\$26.95/TB); egress costs on data extraction
Vendor	Multi-cloud (AWS, Azure, GCP); ANSI SQL portable; open formats	Proprietary lock-in (Time Travel, Snowpipe, Zero-Copy Clone); AI features = 46% of bill

Other	Encryption, RBAC, SSO out of the box; minimal administration	SQL only->no low-code option; no free tier (trial only)
-------	--	---

Vendor Lock-in: Migration Cost If You Leave

- Rebuild all ingestion pipelines
- Recreate security policies & role hierarchy
- Re-implement Time Travel logic elsewhere
- Export all data -> egress costs apply

Strategy: prefer ANSI SQL + open formats by default. Use Snowflake-specific features consciously, not by default.

Compared to Market Alternatives

	Snowflake	BigQuery	Redshift	Synapse
Owner	Independent	Google	Amazon	Microsoft
Setup	Low	Lowest	Medium	High
Scaling	Manual for vertical	Fully auto	Manual	Semi-auto
Pricing	Credits + storage	Per TB scanned	Per node 24/7	Per DWU + storage
Storage/TB (EU)	\$26.95	\$20	\$24 bundled	\$20
Free tier	\$400 / 30 days	1 TB + 10 GB/mo	None	None
Lock-in	Proprietary features	GCP ecosystem	AWS ecosystem	Azure ecosystem

Bottom line: BigQuery is simpler and cheaper for smaller workloads. Snowflake wins when cloud neutrality and multi-source integration matter.

When to Use Snowflake

Use Snowflake When	Avoid When
Growing data volume beyond single DB	OLTP / transactional workloads
Multiple teams need concurrent access	Full infrastructure control required
Multiple disconnected systems to centralize	Already on GCP/AWS -> BigQuery/Redshift cheaper
Low operational overhead needed	Budget too small (BigQuery free tier may suffice)
No existing AWS/Azure/GCP commitment	No SQL knowledge

Cost Structure

Billing Layers (AWS US East, Standard Edition)

Layer	Unit	Rate / Notes
Cloud Services	Credits	Free if < 10% of daily compute
Compute	Credits/second	\$2.00/credit → charged only while running
Storage	GB/month	\$23.00/TB

Credits are Snowflake's unit of measure for compute. Cost = credits consumed × price per credit. **Auto-suspend is critical:** warehouse running 24/7 ≈ \$194/month vs \$117/month with 10h/week. Every resume has a **60-second minimum** charge: avoid frequent short sessions.

Monthly Example: SwissBike SA

Assumptions: AWS US East (N. Virginia, Standard edition, X-Small Gen 2 warehouse (1.35 cr/h), 10h/week, 1 TB storage, CoWork + Cortex AI enabled.

$10\text{h/week} \times 4.333\text{ weeks/month} = 43.33\text{h/month}$
 $43.33\text{h} \times 1.35\text{ cr/h} \times \$2.00/\text{cr} = \$117.00/\text{month}$

Component	Monthly Cost
Compute (XS Gen 2 Warehouse)	\$117.00
Snowflake CoWork	\$123.08
Snowflake Cortex AI (CoCo)	\$5.78
Storage (1 TB)	\$23.00
Total	\$268.86

Key observations: CoWork + Cortex AI = \$128.86/month (48% of total bill) → more than compute itself. Storage is billed on **compressed size**: Snowflake compresses automatically (3–5×). User count does **not** affect cost → multiple users share one warehouse at no extra charge.

How to Get Started

Prerequisites

- Snowflake account (trial: \$400 for 30 days at app.snowflake.com)
- Choose cloud provider (AWS / Azure / GCP) and region → pick same region as your data to avoid egress costs
- Warehouse creation permissions

Setup Steps

- Create a database and schema
- Create a Virtual Warehouse
- Load data (from stage or direct `COPY INTO`)
- Execute queries

Hello-World Example

```
-- 1. Create objects
CREATE DATABASE POC_SNOWFLAKE;
CREATE SCHEMA DEMO;
CREATE WAREHOUSE DEMO_WH
  WITH WAREHOUSE_SIZE = 'XSMALL'
       AUTO_SUSPEND = 60
       AUTO_RESUME = TRUE;

-- 2. Verify session
SELECT CURRENT_ACCOUNT(),
       CURRENT_USER(),
       CURRENT_WAREHOUSE();

-- 3. Query built-in sample data
SELECT * FROM SNOWFLAKE_SAMPLE_DATA
       .TPCH_SF1.CUSTOMER LIMIT 10;
```

Common Operations

Warehouse management

```
ALTER WAREHOUSE COMPUTE_WH SET WAREHOUSE_SIZE = 'SMALL';
ALTER WAREHOUSE COMPUTE_WH SUSPEND;
ALTER WAREHOUSE COMPUTE_WH RESUME;
```

Load data from internal stage

```
PUT file:///data/orders.csv @~/staged;
COPY INTO orders FROM @~/staged
  FILE_FORMAT = (TYPE = 'CSV' SKIP_HEADER = 1);
```

Time Travel — recover dropped table

```
DROP TABLE CUSTOMER_PROD;
UNDROP TABLE CUSTOMER_PROD;
-- Query historical state
SELECT * FROM CUSTOMER_PROD
  AT (OFFSET => -3600); -- 1 hour ago
```

Zero-Copy Clone — safe testing copy

```
CREATE TABLE CUSTOMER_TEST
  CLONE CUSTOMER_PROD;
```

Query profiling

Use **Monitoring** → **Query History** → **Query Profile** to inspect Table Scan, Join, Aggregate and Sort nodes and understand MPP execution.

Vendor lock-in mitigation

Proprietary Feature	Safer Alternative
Snowpipe	<code>COPY INTO</code> or Airbyte
Data Sharing	Export via Parquet / CSV
Zero-Copy Clone	Document dependencies explicitly
Time Travel	Use for recovery only, not logic

Refs: snowflake.com/en/blog/5-reasons-to-love-snowflakes-architecture-for-your-data-warehouse, medium.com/@mastech_digital/snowflake-vs-redshift-vs-bigquery-vs-synapse-a-comprehensive-comparison-dbf36d0f911, snowflake.com/en/pricing-options, snowflake.com/legal-files/CreditConsumptionTable.pdf, docs.snowflake.com/en/guides/overview-cost